Universidad Tecnológica del Perú

JOEDAYZ academy

Introducción a Quarkus

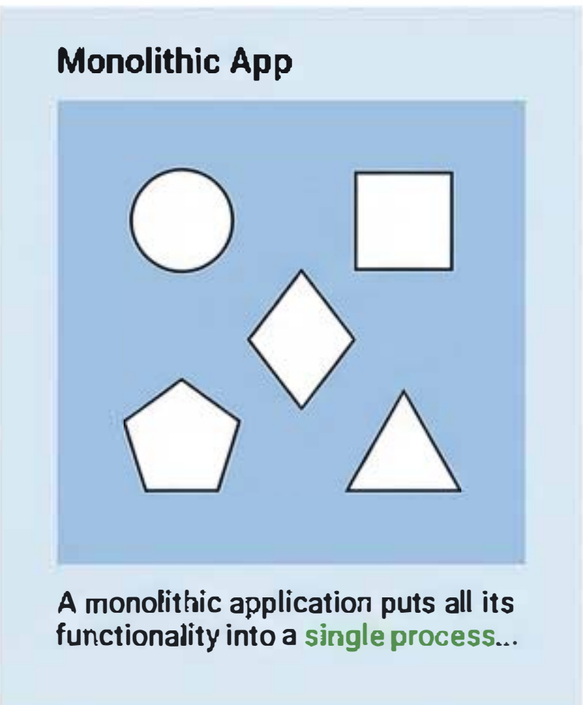
De Monolitos a Microservicios Container-Native

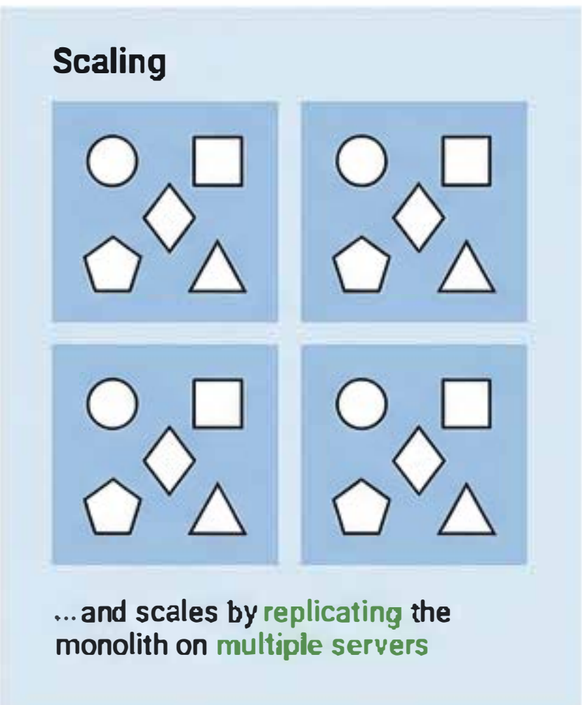
JoeDayz | 2025

La Evolución de la Arquitectura

Aplicaciones Monolíticas:

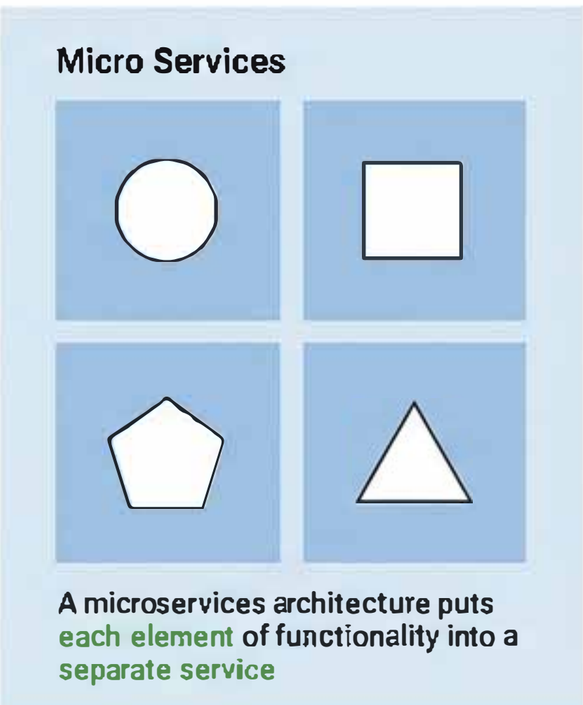
Toda la funcionalidad en un solo proceso. Se escala replicando el monolito completo en múltiples servidores.

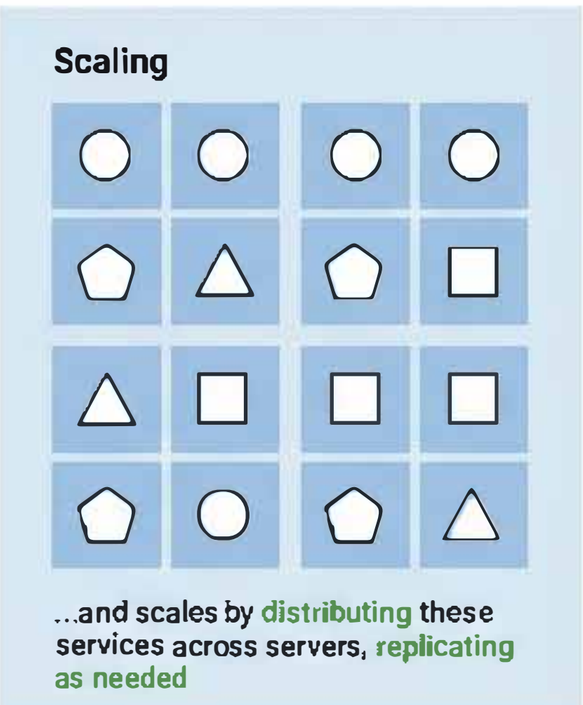




Microservicios:

Cada elemento de funcionalidad en un servicio separado. Modelado por dominios, stateless, y con despliegue independiente.

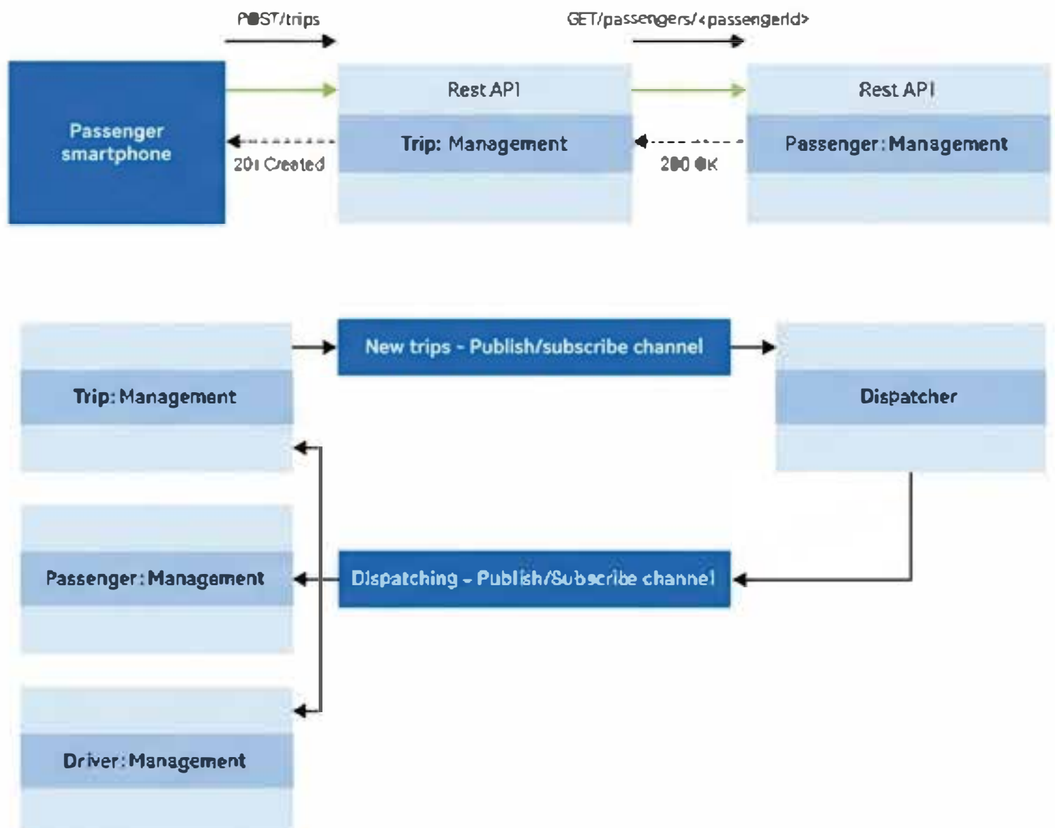




La Complejidad de la Distribución

La arquitectura de microservicios introduce nuevos desafíos operacionales:

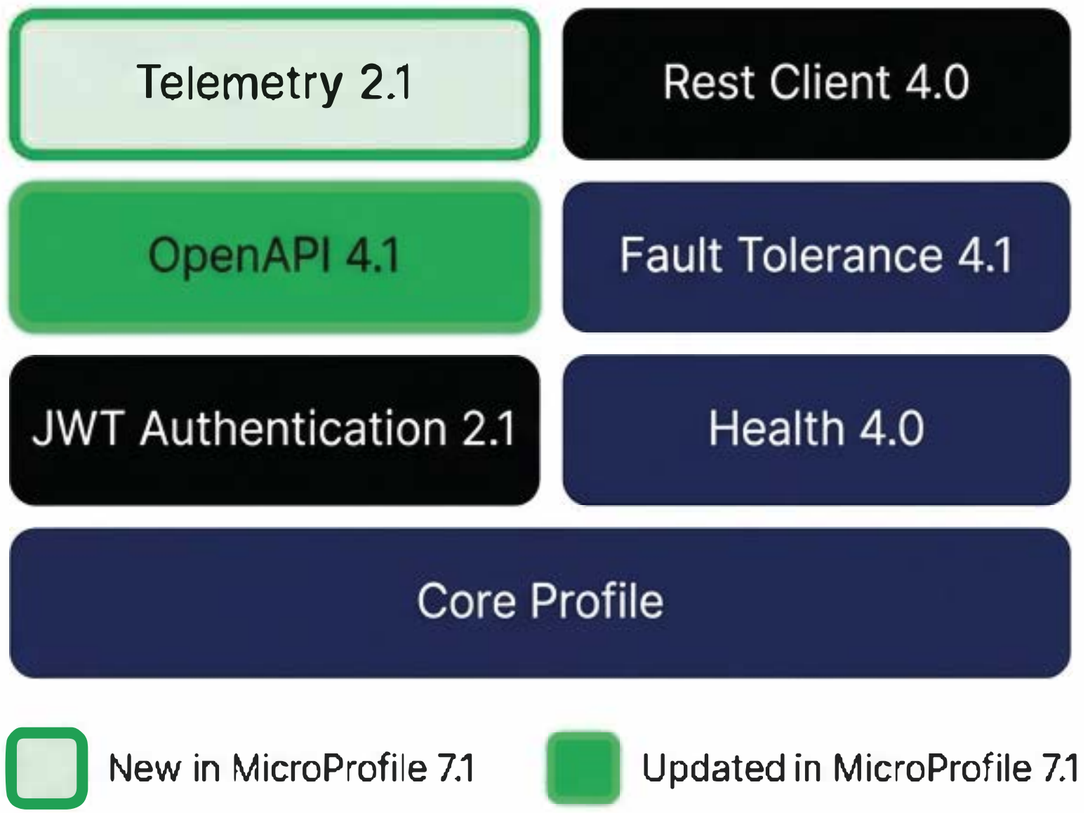
- **Comunicación**: Síncrona (fácil pero bloqueante) vs. Asíncrona (desacoplada pero compleja operativamente).
- **Resiliencia**: Diseñar para fallar mediante Time-out, Retry, Fallback y Circuit Breaker.
- **Observabilidad**: Necesidad de tracing distribuido (OpenTelemetry) y métricas centralizadas (Prometheus, Grafana).
- **Seguridad**: Gestión de Single Sign-on y tokens distribuidos (JWT) mediante un API Gateway.



El Estándar: Eclipse MicroProfile 7.1

Optimización de Java para microservicios y portabilidad entre múltiples runtimes.

- Un joint-venture entre la Fundación Eclipse y proveedores como Red Hat.
- Basado en lo mínimo requerido (JAX-RS, CDI, JSON-P).
- Proporciona un **marco flexible** para **patrones de diseño** comunes en aplicaciones nativas de la nube.



¿Qué es Quarkus?

Un runtime de Java diseñado específicamente para microservicios y entornos container-native.

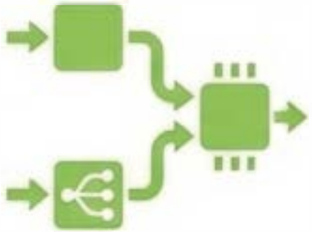
- Surge para mejorar **tiempos de inicio** (startup time) y rendimiento donde implementaciones anteriores fallaron.
- Realiza en **tiempo de compilación** lo que tradicionalmente se hacía en tiempo de ejecución (configuración, arranque, reflexión).
- Cuenta con **soporte empresarial** de Red Hat, asegurando actualizaciones de seguridad y bug fixes constantes.

Filosofía “Developer Joy”

Quarkus está centrado en la experiencia del desarrollador:

Live Reloading: Recarga instantánea de cambios en modo desarrollador sin reiniciar.

Cero Configuración (Zero-config): Un marco obstinado que establece valores predeterminados utilizables y unifica tecnologías en un solo archivo.



Unificación de Modelos: Soporte nativo para programación Imperativa y Reactiva bajo un mismo motor.

Construcción Integrada: Generación nativa sin fricción para todas las bibliotecas incluidas.

Implementando el Estándar con SmallRye

Quarkus implementa las especificaciones de MicroProfile a través de los proyectos integrados de SmallRye:

- Tolerancia a Fallos (`smallrye-fault-tolerance`)
- Métricas y Health Checks (`smallrye-metrics`, `smallrye-health`)
- Seguridad JWT (`smallrye-jwt`)
- (`smallrye-reactive-messaging`)

MicroProfile Specification	Project Name	Quarkus Dependency (Maven group:artifact)
Rest Client	SmallRye REST Client	`io.quarkus:quarkus-smallrye-rest-client`
Fault Tolerance	SmallRye Fault Tolerance	`io.quarkus:quarkus-smallrye-fault-tolerance`
Health Check	SmallRye Health	`io.quarkus:quarkus-smallrye-health`
Metrics	SmallRye Metrics	`io.quarkus:quarkus-smallrye-metrics`
JWT Security	SmallRye JWT	`io.quarkus:quarkus-smallrye-jwt`
OpenAPI	SmallRye OpenAPI	`io.quarkus:quarkus-smallrye-openapi`
OpenTelemetry	Quarkus OpenTelemetry	`io.quarkus:quarkus-opentelemetry`
Reactive Streams Operators	SmallRye Reactive Streams Operators	`io.quarkus:quarkus-smallrye-reactive-streams-operators`
Reactive Streams Messaging	SmallRye Reactive Messaging	`io.quarkus:quarkus-smallrye-reactive-messaging`

El Desafío Cloud: Container-Native

En cargas de trabajo Serverless y Functions as a Service (FaaS), la JVM tradicional es demasiado pesada. Los requerimientos críticos son:

1. **Tiempos de inicio ultrarrápidos (Fast startup).**
2. **Huella de memoria mínima (Small memory footprint).**
3. **Uso reducido de disco (Smaller disk usage).**

> "Incluso un tiempo de inicio de un segundo puede ser demasiado en la nube moderna."

El Motor de Rendimiento: GraalVM

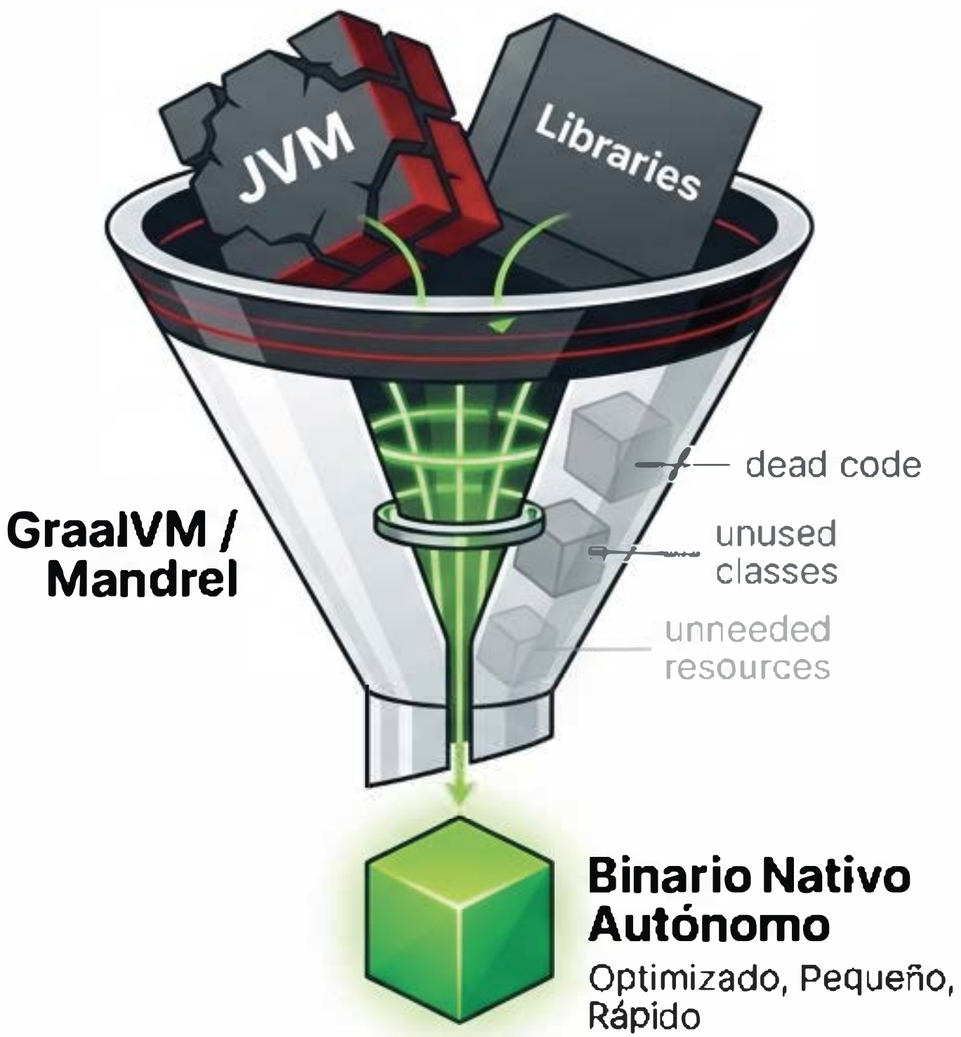
Para cumplir con las exigencias del cloud, Quarkus ofrece soporte total para compilación anticipada (AoT) usando GraalVM.

- **Compilación AoT**: GraalVM compila la aplicación Java directamente a un ejecutable **nativo** de la plataforma.
- La **Suposición del Mundo Cerrado (Closed-world assumption)**: La JVM necesita saber exactamente qué clases se utilizarán en tiempo de compilación. Quarkus resuelve esto evaluando la reflexión anticipadamente.

GraalVM™

Compilación Nativa en Acción

- **Eliminación de Código Muerto:** Durante la compilación AoT, GraalVM elimina todas las clases innecesarias de la aplicación, las librerías dependientes e incluso de la JVM.
- **Mandrel:** Red Hat proporciona esta distribución específica de GraalVM enfocada exclusivamente en generar imágenes nativas optimizadas para aplicaciones Quarkus.
- **Resultado**: Un binario autónomo que no requiere una JVM instalada en el contenedor para ejecutarse.

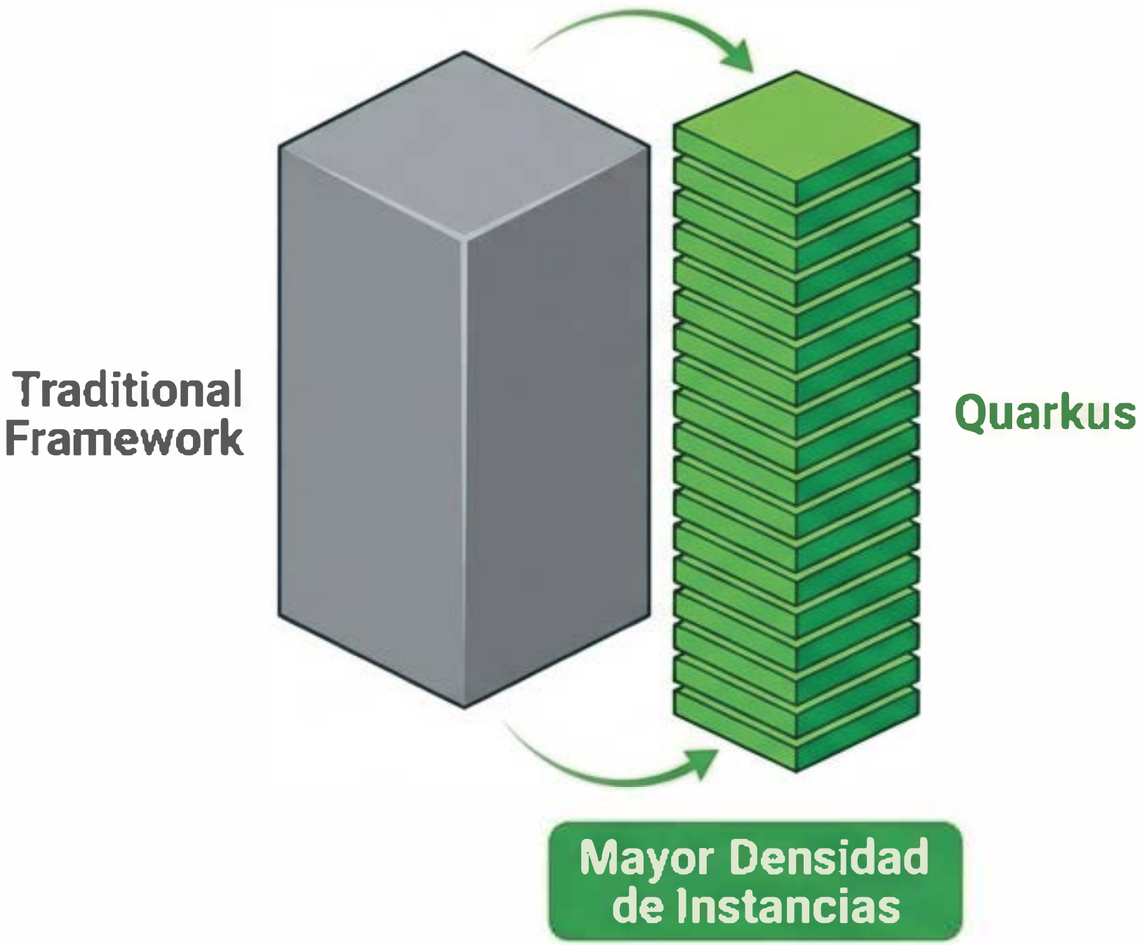


Ventajas Competitivas

Comparado con los frameworks cloud-native basados en Java tradicionales, Quarkus ofrece una densidad de despliegue muy superior:

- **Tiempo de respuesta** inicial casi instantáneo.
- **Consumo de memoria** radicalmente inferior.
- **Alto rendimiento** en rutas reactivas.
- **Soporte nativo** de **contenedores** de primera clase.
- Cumplimiento total con los estándares de MicroProfile.

Densidad de Despliegue



La Evolución de Spring hacia GraalVM

El ecosistema Spring también reconoció la necesidad de adaptarse al paradigma Container-Native:

- **2019-2022: Proyectos experimentales separados (spring-native)**

  Requerían configuraciones manuales complejas y dependían mucho de hints de reflexión manuales.

- **2023 (Spring Boot 3.0): El gran salto**

  Integración oficial de GraalVM Native Images y la introducción del AOT Engine directamente en Spring Framework 6.

2019

Empiezan experimentos con GraalVM

2020-2021

Pivotal/VMware Tanzu lanza Spring Native Experimental (soporte limitado con Spring Boot 2.3+)

2022-2023 (Nov 2022)

Spring Boot 3.0: Integración oficial, AOT Engine en Framework 6, eliminación de **spring-native**. Requiere GraalVM 22.x+ y Java 17+

El Paisaje Actual: Spring Boot 3.3.x

Hoy en día, Spring Boot ofrece soporte sólido y out-of-the-box para compilación nativa:

- **Rendimiento**: Arranques en <100ms y uso de memoria muy bajo.
- **Cloud Native Buildpacks (CNB)**: Herramientas como Paketo convierten la aplicación en imágenes de contenedor nativas sin necesidad de escribir un Dockerfile manual o instalar GraalVM localmente.
- **Competencia Directa**: Ambos frameworks ahora apuntan al mismo objetivo Serverless, pero Quarkus mantiene la ventaja de haber nacido con ADN nativo y estándares MicroProfile.

bash

```bash
./mvnw spring-boot:build-image
```

- **2025 (Hoy)**
- **Spring Boot 3.3.x** (última estable, que tú ya usas 😉).
  - Soporte out-of-the-box para GraalVM Native Image.
  - Funciona con Java 21 y GraalVM 24.x.
  - Se recomienda usar **Spring AOT Engine** en lugar de configuraciones manuales.
  - La comunidad está reportando aplicaciones con arranque en <100ms y uso de memoria muy bajo, ideales para serverless, microservicios y apps cloud-native.

Ecosistema y Entorno de Desarrollo

Herramientas recomendadas para el curso introductorio:

- **Inicialización**: code.quarkus.io para generar la estructura del proyecto Maven/Gradle.
- **Construcción**: Apache Maven y JDK 21 + (GraalVM).
- **Contenedores**: Docker Desktop o Podman Desktop para pruebas locales de imágenes nativas.
- **IDE**: VSCode o IntelliJ IDEA con soporte para Java.

Siguientes Pasos

- ✓ Revisa la documentación oficial de Red Hat Build of Quarkus.
- ✓ Genera tu primer proyecto REST en code.quarkus.io.
- ✓ Prepara tu entorno local instalando GraalVM y un motor de contenedores (Docker/Podman).